Map api : 👍

https://www.mapsdirections.info/crear-un-mapa-de-google/

Images/audio/video 👍

https://pixabay.com/

https://unsplash.com/

Icons 👍

https://boxicons.com/

https://icons.getbootstrap.com/

https://ionic.io/ionicons

https://www.flaticon.com

https://react-icons.github.io/react-icons/

React

https://www.npmjs.com/package/react-router-dom

https://react-bootstrap.netlify.app/docs/getting-started/introduction

https://tailwindcss.com/docs/installation/using-vite

https://www.npmjs.com/package/axios

https://www.npmjs.com/package/react-icons

https://www.npmjs.com/package/dotenv

https://www.npmjs.com/package/jsonwebtoken

https://www.npmjs.com/package/bcrypt

JavaScript Storage

- localStorage
- sessionStorage
- Cookies

localStorage

✅ Stores data in browser permanently
✅ No expiration time
✅ Data remains after browser close
✅ Storage limit ≈ 5MB
✅ Stores only **strings**
❌ Not sent to server automatically

```
localStorage.setItem("key", "value");
localStorage.getItem("key");
localStorage.removeItem("key");
localStorage.clear();

localStorage.setItem("user", JSON.stringify(obj));
JSON.parse(localStorage.getItem("user"));


// Store Data
localStorage.setItem("name", "Ganapathi");

// Get Data
let data = localStorage.getItem("name");
console.log(data);

// Remove Specific Item
localStorage.removeItem("name");

// Clear All
localStorage.clear();



let user = {
  name: "Ganapathi",
  age: 25
};

localStorage.setItem("user", JSON.stringify(user));
```

```
let storedUser = JSON.parse(localStorage.getItem("user"));
console.log(storedUser.name);
```

sessionStorage

✅ Same API as localStorage
✅ Data removed when tab/browser closes
✅ Used for temporary data
✅ Storage limit ≈ 5MB
❌ Not sent to server

```
sessionStorage.setItem("email", "test@gmail.com");

let email = sessionStorage.getItem("email");
console.log(email);

sessionStorage.removeItem("email");
```

Cookies

✅ Small data storage (~4KB)
✅ Can set expiry time
✅ Automatically sent to server with every request
✅ Used for authentication/session

```
document.cookie = "username=Ganapathi; expires=Fri, 31 Dec 2026 12:00:00 UTC; path=/";


console.log(document.cookie);


document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC;";
```

Feature	localStorage	sessionStorage	Cookies
Expiry	No	Tab Close	Custom
Size	~5MB	~5MB	~4KB

Sent to Server	❌	❌	✅
Use Case	Settings	Temporary Data	Login/Auth

```
// Common JavaScript Basic Programs (Default Interview Programs)
// 1) Hello World
console.log("Hello World");
// 2) Add Two Numbers
let a = 10;
let b = 20;
let sum = a + b;
console.log("Sum is:", sum);
// 3) Swap Two Numbers (Using Third Variable)
let a = 5;
let b = 10;
let temp = a;
a = b;
b = temp;
console.log("a:", a, "b:", b);
// 4) Check Even or Odd
let num = 7;
if(num % 2 === 0){
 console.log("Even");
}else{
 console.log("Odd");
}
// 5) Positive / Negative / Zero
let num = -5;
if(num > 0){
 console.log("Positive");
}else if(num < 0){
 console.log("Negative");
}else{
 console.log("Zero");
}
// 6) Largest of Two Numbers
let a = 20;
let b = 15;
```

```
if(a > b){
 console.log("A is largest");
}else{
 console.log("B is largest");
}
// 7) Largest of Three Numbers
let a = 10, b = 25, c = 15;
if(a > b && a > c){
 console.log("A is largest");
}else if(b > c){
 console.log("B is largest");
}else{
 console.log("C is largest");
}
// 8) Reverse a Number
let num = 1234;
let reverse = 0;
while(num > 0){
 let digit = num % 10;
 reverse = reverse * 10 + digit;
 num = Math.floor(num / 10);
}
console.log("Reverse:", reverse);
// 9) Reverse a String
let str = "javascript";
let reversed = str.split("").reverse().join("");
console.log(reversed);
// 10) Palindrome Number
let num = 121;
let original = num;
let reverse = 0;
while(num > 0){
 let digit = num % 10;
 reverse = reverse * 10 + digit;
 num = Math.floor(num / 10);
}
if(original === reverse){
 console.log("Palindrome");
}else{
 console.log("Not Palindrome");
```

```
}
// 11) Factorial
let num = 5;
let fact = 1;
for(let i = 1; i <= num; i++){
 fact *= i;
}
console.log("Factorial:", fact);
// 12) Fibonacci Series
let n = 10;
let a = 0, b = 1;
console.log(a);
console.log(b);
for(let i = 2; i < n; i++){
 let next = a + b;
 console.log(next);
 a = b;
 b = next
}


// 13) Prime Number Check
let num = 7;
let isPrime = true;
if(num <= 1){
 isPrime = false;
}
for(let i = 2; i < num; i++){
 if(num % i === 0){
 isPrime = false;
 break;
 }
}
if(isPrime){
 console.log("Prime");
}else{
 console.log("Not Prime");
}
// 14) Sum of Array
let arr = [10, 20, 30, 40];
```

```
let sum = 0;
for(let i = 0; i < arr.length; i++){
 sum += arr[i];
}
console.log("Sum:", sum);
// 15) Largest in Array
let arr = [5, 10, 25, 3];
let max = arr[0];
for(let i = 1; i < arr.length; i++){
 if(arr[i] > max){
 max = arr[i];
 }
}
console.log("Largest:", max);



// JavaScript Pattern, String, and Array Programs
// ==============================
// PATTERN PROGRAMS
// ==============================
// 1) Right Triangle Star Pattern
*
**
***
****
*****
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let j = 1; j <= i; j++){
 pattern += "*";
 }
 console.log(pattern);
}
// 2) Reverse Right Triangle
*****
****
***
```

```
**
*
Code:
let n = 5;
for(let i = n; i >= 1; i--){
 let pattern = "";
 for(let j = 1; j <= i; j++){
 pattern += "*";
 }
 console.log(pattern);
}
// 3) Number Pattern
1
12
123
1234
12345
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let j = 1; j <= i; j++){
 pattern += j;
 }
 console.log(pattern);
}
// 4) Pyramid Pattern
 *
 ***
 *****
 *******
*********
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let space = 1; space <= n - i; space++){
 pattern += " ";
 }
 for(let star = 1; star <= (2*i - 1); star++){
```

```
 pattern += "*";
 }
 console.log(pattern
}

// ==============================
// STRING PROGRAMS
// ==============================
// 1) Reverse String
let str = "javascript";
let reversed = "";
for(let i = str.length - 1; i >= 0; i--){
 reversed += str[i];
}
console.log(reversed);
// 2) Palindrome Check
let str = "madam";
let reversed = str.split("").reverse().join("");
if(str === reversed){
 console.log("Palindrome");
}else{
 console.log("Not Palindrome");
}
// 3) Count Vowels
let str = "javascript";
let count = 0;
for(let i = 0; i < str.length; i++){
 let ch = str[i].toLowerCase();
 if(ch === 'a' || ch === 'e' || ch === 'i' || ch === 'o' || ch === 'u'){
 count++;
 }
}
console.log("Vowel count:", count);
// 4) Character Frequency
let str = "hello";
let obj = {};
for(let char of str){
 if(obj[char]){
 obj[char]++;
 }else{
```

```
 obj[char] = 1;
 }
}
console.log(obj);
// 5) Remove Duplicates from String
let str = "programming";
let result = "";
for(let char of str){
 if(!result.includes(char)){
 result += char;
 }
}
console.log(result);




// ==============================
// ARRAY PROGRAMS
// ==============================
// 1) Find Largest Element
let arr = [10, 45, 22, 98, 5];
let max = arr[0];
for(let i = 1; i < arr.length; i++){
 if(arr[i] > max){
 max = arr[i];
 }
}
console.log("Largest:", max);
// 2) Second Largest Element
let arr = [10, 45, 22, 98, 5];
let first = -Infinity;
let second = -Infinity;
for(let num of arr){
 if(num > first){
 second = first;
 first = num;
 } else if(num > second && num !== first){
 second = num;
```

```
 }
}
console.log("Second Largest:", second);
// 3) Remove Duplicates from Array
let arr = [1,2,3,2,4,1,5];
let unique = [];
for(let num of arr){
 if(!unique.includes(num)){
 unique.push(num);
 }
}
console.log(unique);
// 4) Reverse Array
let arr = [1,2,3,4,5];
let reversed = [];
for(let i = arr.length - 1; i >= 0; i--){
 reversed.push(arr[i]);
}
console.log(reversed);
// 5) Sum of Array
let arr = [10,20,30,40];
let sum = 0;
for(let num of arr){
 sum += num;
}
console.log("Sum:", sum)


for(let i =0 ;i<5; i++){

    }

    initi ==> declaration ==> definition ==>  starting ==> excuting value
    check ==> condition check ==> lenth of condition ==> operation
    inc/Dec ==> itration ==> count itration
```

Html 👍

Elements & Attributes ⇒ 2

Tags ⇒ 2

- Basic Tags ⇒ 6
- Major ⇒ 10
- Multimedia tags ⇒ 6
- Semitic Tags ⇒ 12
- Dom & API

Css 👍

- Importing methods
- Selectors
- Typography
- Dimension
- Units
- Box
- Background
- Z index
- Position
- Flex
- Grid
- Media
- Responsive ness
- Color 3 methods

Js 👍

- Variable
- Data type
  - Primitive & non primitive
- Operator
- Statements
  - Condition
  - Looping
- Date and Math object
- Operations
  - String
  - Array
- Dom Manipulating elements
- ES6 ⇒
  - Fetching
  - Async await
  - Promise methods
  - Scheduling methods
  - Model modules
  - ‘useStrict’
  - Template string

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Modern Dynamic Webpage</title>
    <style>
        :root {
            --primary: #2563eb;
            --primary-dark: #1d4ed8;
            --text: #1f2937;
            --bg: #f9fafb;
            --card-bg: #ffffff;
            --border: #e5e7eb;
            --danger: #dc2626;
            --success: #16a34a;
        }

        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
            font-family: -apple-system, BlinkMacSystemFont, "Segoe
UI", Roboto, sans-serif;
        }

        body {
            background-color: var(--bg);
            color: var(--text);
            line-height: 1.6;
        }

        header {
            display: flex;
```

```
        justify-content: space-around;
        align-items: center;
        background-color: #4b5563;
        padding: 10px 20px;
        box-shadow: inset 4px -4px 5px 14px #39445491;
    }

    header .logo {
        padding: 10px;
        color: var(--primary);
        text-transform: capitalize;
        display: flex;
        justify-content: space-around;
        align-items: center;
    }

    .logo a {
        text-decoration: none;
        color: var(--card-bg);
    }

    .logo span {
        color: var(--success);
        font-size: 23px;
        font-style: italic;
    }

    .logo .mobileicon {
        font-size: 28px;
        padding: 10px;
        display: none;
    }

    .menus ul li {
        list-style-type: none;
        display: inline-block;
        padding: 10px;
    }

    .menus a {
```

```
        text-decoration: none;
        font-size: 18px;
        color: var(--bg);
    }

    .menus ul li:hover a {
        color: var(--success);
        font-weight: 500;
    }


    /* Layout & Components */
    .section-container {
        max-width: 1000px;
        margin: 0 auto;
        padding: 60px 20px;
    }

    .section-title {
        font-size: 2rem;
        font-weight: 700;
        text-align: center;
        margin-bottom: 30px;
    }

    /* Hero */
    #hero {
        min-height: 75vh;
        display: flex;
        flex-direction: column;
        justify-content: center;
        align-items: center;
        text-align: center;
        background: linear-gradient(135deg, #1e293b, #0f172a);
        color: #ffffff;
        padding: 40px 20px;
    }

    #hero h1 {
        font-size: 2.8rem;
```

```
    margin-bottom: 16px;
}

#hero p {
    font-size: 1.2rem;
    color: #94a3b8;
    max-width: 600px;
    margin-bottom: 24px;
}

.btn {
    padding: 12px 28px;
    border-radius: 6px;
    border: none;
    background: var(--primary);
    color: #fff;
    font-size: 1rem;
    cursor: pointer;
    transition: background 0.2s;
}

.btn:hover {
    background: var(--primary-dark);
}

/* Slider */
.slider-wrapper {
    position: relative;
    overflow: hidden;
    border-radius: 12px;
    background: #000;
    height: 380px;
}

.slider-track {
    display: flex;
    height: 100%;
    transition: transform 0.4s ease-in-out;
}
```

```css
.slide-card {
    min-width: 100%;
    height: 100%;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    padding: 40px;
    background: #ffffff;
    text-align: center;
    box-shadow: inset 0 0 20px rgba(0, 0, 0, 0.05);
}

.slide-card h3 {
    font-size: 1.8rem;
    color: var(--primary);
    margin-bottom: 12px;
}

.slide-card p {
    max-width: 500px;
    color: #4b5563;
}

.slider-controls {
    display: flex;
    justify-content: space-between;
    position: absolute;
    width: 100%;
    top: 50%;
    transform: translateY(-50%);
    padding: 0 16px;
}

.slider-arrow {
    background: rgba(0, 0, 0, 0.6);
    color: white;
    border: none;
    width: 40px;
    height: 40px;
```

```
            border-radius: 50%;
            cursor: pointer;
            font-size: 1.2rem;
        }

        .slider-dots {
            display: flex;
            justify-content: center;
            gap: 8px;
            margin-top: 16px;
        }

        .dot {
            width: 12px;
            height: 12px;
            border-radius: 50%;
            background: #cbd5e1;
            cursor: pointer;
        }

        .dot.active {
            background: var(--primary);
        }

        /* FAQ Accordion */
        .faq-item {
            background: var(--card-bg);
            border: 1px solid var(--border);
            border-radius: 6px;
            margin-bottom: 12px;
            overflow: hidden;
        }

        .faq-question {
            padding: 18px 20px;
            display: flex;
            justify-content: space-between;
            cursor: pointer;
            font-weight: 600;
            user-select: none;
```

```
}

.faq-answer {
    max-height: 0;
    overflow: hidden;
    transition: max-height 0.3s ease;
    padding: 0 20px;
    color: #4b5563;
}

.faq-item.active .faq-answer {
    padding: 0 20px 18px 20px;
}

.faq-icon {
    transition: transform 0.3s ease;
}

.faq-item.active .faq-icon {
    transform: rotate(45deg);
}

/* Form */
.form-box {
    background: var(--card-bg);
    border: 1px solid var(--border);
    padding: 32px;
    border-radius: 8px;
    max-width: 550px;
    margin: 0 auto;
}

.form-group {
    margin-bottom: 20px;
    display: flex;
    flex-direction: column;
}

.form-group label {
    font-size: 0.9rem;
```

```
            font-weight: 600;
            margin-bottom: 6px;
        }

        .form-group input,
        .form-group textarea {
            padding: 10px 14px;
            border: 1px solid var(--border);
            border-radius: 6px;
            outline: none;
            font-size: 0.95rem;
        }

        .form-group.error input,
        .form-group.error textarea {
            border-color: var(--danger);
        }

        .form-group.success input,
        .form-group.success textarea {
            border-color: var(--success);
        }

        .error-msg {
            color: var(--danger);
            font-size: 0.8rem;
            margin-top: 4px;
            display: none;
        }

        .form-group.error .error-msg {
            display: block;
        }

        /* Footer */
        footer {
            background: #0f172a;
            color: #94a3b8;
            padding: 40px 20px;
            text-align: center;
```

```
                border-top: 1px solid #1e293b;
            }

            .geo-box {
                margin-top: 16px;
                display: inline-block;
                padding: 8px 16px;
                background: #1e293b;
                border-radius: 20px;
                font-size: 0.9rem;
            }
        </style>
    </head>

    <body>
        <header>
            <div class="logo">
                <a href="#">
                    <h3>logo <span>'s</span></h3>
                </a>
                <div class="mobileicon">
                    ☰
                </div>
            </div>
            <div class="menus">
                <ul id="menus">
                    <li><a href="#"> Home</a></li>
                    <li><a href="#"> About</a></li>
                    <li><a href="#"> Service</a></li>
                    <li><a href="#"> Contact</a></li>
                </ul>
            </div>
        </header>
        <section id="hero">
            <h1 id="hero-title"></h1>
            <p id="hero-desc"></p>
            <button class="btn" id="hero-btn">Explore Features</button>
        </section>

        <section class="section-container" id="slider-section">
```

```
            <h2 class="section-title">Core Highlights</h2>
            <div class="slider-wrapper">
                <div class="slider-track" id="slider-track"></div>
                <div class="slider-controls">
                    <button class="slider-arrow"
id="prev-btn">&#10094;</button>
                    <button class="slider-arrow"
id="next-btn">&#10095;</button>
                </div>
            </div>
            <div class="slider-dots" id="slider-dots"></div>
        </section>

        <section class="section-container" id="faq-section">
            <h2 class="section-title">Frequently Asked Questions</h2>
            <div id="faq-list"></div>
        </section>

        <section class="section-container" id="contact-section">
            <h2 class="section-title">Get in Touch</h2>
            <div class="form-box">
                <form id="contact-form" novalidate>
                    <div class="form-group" id="name-group">
                        <label for="name">Full Name</label>
                        <input type="text" id="name" placeholder="John
Doe" />
                        <span class="error-msg">Name must be at least 3
characters long.</span>
                    </div>

                    <div class="form-group" id="email-group">
                        <label for="email">Email Address</label>
                        <input type="email" id="email"
placeholder="john@example.com" />
                        <span class="error-msg">Enter a valid email
address.</span>
                    </div>

                    <div class="form-group" id="phone-group">
                        <label for="phone">Phone Number</label>
```

```
                        <input type="tel" id="phone"
placeholder="10-digit number" />
                        <span class="error-msg">Enter a valid 10-digit
mobile number.</span>
                    </div>

                    <div class="form-group" id="message-group">
                        <label for="message">Message</label>
                        <textarea id="message" rows="4" placeholder="How
can we help?"></textarea>
                        <span class="error-msg">Message must be at least
10 characters long.</span>
                    </div>

                    <button type="submit" class="btn" style="width:
100%;">Send Message</button>
                </form>
            </div>
        </section>

        <footer>
            <p>&copy; 2026 Core Vanilla Project. All rights
reserved.</p>
            <div class="geo-box" id="geo-display">Detecting
location...</div>
        </footer>

        <script>

            const menusItem = [
                { mId: 1, menuName: "Home" },
                { mId: 2, menuName: "About" },
                { mId: 3, menuName: "Service" },
                { mId: 4, menuName: "Contact" },
            ];
            menusItem.map((items) => {
                let Items = ''
                let listItem = document.getElementById("menus");
                Items += `
```

```
            <li ${items.mId}>  <a href="#" > ${items.menuName} </a>
</li>
          `;

                listItem.innerHTML = items;

            })
            //
------------------------------------------------------------
            // 1. HERO SECTION ENGINE
            //
------------------------------------------------------------
            const heroData = {
                title: "Engineered With Pure Vanilla JavaScript",
                desc: "Zero external dependencies, zero animation
libraries. Dynamic rendering and full DOM control directly from script."
            };

            document.getElementById("hero-title").textContent =
heroData.title;
            document.getElementById("hero-desc").textContent =
heroData.desc;

document.getElementById("hero-btn").addEventListener("click", () => {

document.getElementById("slider-section").scrollIntoView({ behavior:
"smooth" });
            });

            //
------------------------------------------------------------
            // 2. SLIDER CARD ENGINE
            //
------------------------------------------------------------
            const slides = [
                { title: "Zero Dependencies", text: "Optimized
performance by executing logic with direct browser APIs." },
                { title: "Dynamic State UI", text: "Every card and
accordion node is programmatically rendered." },
```

```
                { title: "Robust Form Handling", text: "Real-time regex
feedback before processing submission events." }
            ];

            const track = document.getElementById("slider-track");
            const dotsContainer =
document.getElementById("slider-dots");
            let currentSlide = 0;
            let autoSlideInterval;

            slides.forEach((item, index) => {
                const slide = document.createElement("div");
                slide.className = "slide-card";
                slide.innerHTML =
`<h3>${item.title}</h3><p>${item.text}</p>`;
                track.appendChild(slide);

                const dot = document.createElement("span");
                dot.className = `dot ${index === 0 ? "active" : ""}`;
                dot.addEventListener("click", () => goToSlide(index));
                dotsContainer.appendChild(dot);
            });

            function updateSlider() {
                track.style.transform = `translateX(-${currentSlide *
100}%)`;
                document.querySelectorAll(".dot").forEach((d, idx) => {
                    d.classList.toggle("active", idx === currentSlide);
                });
            }

            function goToSlide(idx) {
                currentSlide = idx;
                updateSlider();
                resetSliderTimer();
            }

            function nextSlide() {
                currentSlide = (currentSlide + 1) % slides.length;
                updateSlider();
```

```
            }

            function prevSlide() {
                currentSlide = (currentSlide - 1 + slides.length) %
slides.length;
                updateSlider();
            }

            function resetSliderTimer() {
                clearInterval(autoSlideInterval);
                autoSlideInterval = setInterval(nextSlide, 4500);
            }


document.getElementById("next-btn").addEventListener("click", () => {
nextSlide(); resetSliderTimer(); });

document.getElementById("prev-btn").addEventListener("click", () => {
prevSlide(); resetSliderTimer(); });
            resetSliderTimer();

            //
------------------------------------------------------------
            // 3. FAQ ACCORDION ENGINE
            //
------------------------------------------------------------
            const faqs = [
                { q: "Are any frameworks or CDNs imported?", a: "No. All
components, styling hooks, and behaviors run entirely on standard browser
APIs." },
                { q: "How does the form validation run?", a: "Custom
regular expressions run on both real-time input events and submit
verification." },
                { q: "How does the footer detect location?", a: "It
requests user access via `navigator.geolocation` and computes coordinates
without any map SDKs." }
            ];

            const faqList = document.getElementById("faq-list");
```

```
            faqs.forEach((faq) => {
                const item = document.createElement("div");
                item.className = "faq-item";
                item.innerHTML = `
            <div class="faq-question">
              <span>${faq.q}</span>
              <span class="faq-icon">+</span>
            </div>
            <div class="faq-answer">
              <p>${faq.a}</p>
            </div>
          `;

                const questionHead =
item.querySelector(".faq-question");
                const answerBody = item.querySelector(".faq-answer");

                questionHead.addEventListener("click", () => {
                    const isActive = item.classList.contains("active");

                    // Close all open FAQs
                    document.querySelectorAll(".faq-item").forEach((el)
=> {
                        el.classList.remove("active");
                        el.querySelector(".faq-answer").style.maxHeight
= null;
                    });

                    // Toggle clicked FAQ
                    if (!isActive) {
                        item.classList.add("active");
                        answerBody.style.maxHeight =
answerBody.scrollHeight + "px";
                    }
                });

                faqList.appendChild(item);
            });
```

```
            //
------------------------------------------------------------
            // 4. FORM VALIDATION ENGINE
            //
------------------------------------------------------------
            const form = document.getElementById("contact-form");
            const inputs = {
                name: document.getElementById("name"),
                email: document.getElementById("email"),
                phone: document.getElementById("phone"),
                message: document.getElementById("message")
            };

            const rules = {
                name: (val) => val.trim().length >= 3,
                email: (val) =>
/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(val.trim()),
                phone: (val) => /^[0-9]{10}$/.test(val.trim()),
                message: (val) => val.trim().length >= 10
            };

            function validateField(name) {
                const field = inputs[name];
                const parent = field.parentElement;
                const isValid = rules[name](field.value);

                if (!isValid) {
                    parent.classList.add("error");
                    parent.classList.remove("success");
                } else {
                    parent.classList.remove("error");
                    parent.classList.add("success");
                }
                return isValid;
            }

            // Attach real-time validation listeners
            Object.keys(inputs).forEach((key) => {
                inputs[key].addEventListener("input", () =>
validateField(key));
```

```
                inputs[key].addEventListener("blur", () =>
validateField(key));
            });

            form.addEventListener("submit", (e) => {
                e.preventDefault();
                let formIsValid = true;

                Object.keys(inputs).forEach((key) => {
                    const isValid = validateField(key);
                    if (!isValid) formIsValid = false;
                });

                if (formIsValid) {
                    alert("Form submitted successfully!");
                    form.reset();
                    Object.keys(inputs).forEach((key) => {

inputs[key].parentElement.classList.remove("success", "error");
                    });
                }
            });

            //
------------------------------------------------------------
            // 5. GEOLOCATION FOOTER ENGINE
            //
------------------------------------------------------------
            const geoDisplay = document.getElementById("geo-display");

            function fetchLocation() {
                if (!("geolocation" in navigator)) {
                    geoDisplay.textContent = "Geolocation is not
supported by your browser.";
                    return;
                }

                navigator.geolocation.getCurrentPosition(
                    (position) => {
                        const lat = position.coords.latitude.toFixed(4);
```

```
                        const lon =
position.coords.longitude.toFixed(4);
                        geoDisplay.textContent = `User Coordinates: Lat
${lat}°, Lon ${lon}°`;
                    },
                    (error) => {
                        switch (error.code) {
                            case error.PERMISSION_DENIED:
                                geoDisplay.textContent = "Location
access denied by user.";
                                break;
                            case error.POSITION_UNAVAILABLE:
                                geoDisplay.textContent = "Location
information is unavailable.";
                                break;
                            case error.TIMEOUT:
                                geoDisplay.textContent = "Request to
obtain location timed out.";
                                break;
                            default:
                                geoDisplay.textContent = "Unable to
retrieve location.";
                        }
                    },
                    { enableHighAccuracy: true, timeout: 8000 }
                );
            }

            window.addEventListener("DOMContentLoaded", fetchLocation);
        </script>
    </body>

    </html>
```

```
    <!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="assets/logo.png" type="image/x-icon">
    <title>Co!Workers</title>
    <link rel="stylesheet" href="style.css">
    <style>
        table {
            border-collapse: collapse;
        }

        table,
        tr,
        th,
        td {
            border: 1px solid red;
            padding: 10px;
        }
    </style>
</head>

<body>
    <nav class="myMenubar">
        <div class="logo">
            <div class="imglogo">
                <img src="assets/logo.png" alt="logo">
            </div>
            <div class="mobileicon">
                <ion-icon name="menu-outline"></ion-icon>
            </div>
        </div>
        <div class="navbar">
            <ul>
                <li><a href="#">Home</a></li>
                <li><a href="about.html">About</a></li>
                <li><a href="#">Service</a></li>
                <li class="supermenu"><a href="#">Teams</a>
                    <ul class="submenus">
                        <li><a href="#">Software Team</a></li>
                        <li><a href="#">Hardware Team</a></li>
                        <li><a href="#">Testing Team</a></li>
                    </ul>
```

```
            </li>
            <li><a href="contact.html">Contact</a></li>
        </ul>
    </div>
</nav>

<section class="addcart">
    <button onclick="decCount()">-</button>
    <button id="count">0</button>
    <button onclick="addCount()">+</button>

    <!-- <div id="datas"></div> -->

    <!-- <div>
        <ul id="datas">

        </ul>
    </div> -->

    <div>
        <table id="datas">
            <tr>
                <th>d.id</th>
                <th>d.Title</th>
            </tr>
        </table>
    </div>
    <script>
        var incCount = 0;
        let oldCount = document.getElementById("count");
        function updateCount() {
            oldCount.textContent = incCount;
        };

        function addCount() {
            if (incCount <= 5) {
                // incCount++;
                incCount = incCount + 1;
                updateCount()
            }
```

```
        };

        const decCount = () => {
            if (incCount > 1) {
                incCount--;
                updateCount()
            }
        }


        // localStorage.setItem("user", JSON.stringify(obj));
        // JSON.parse(localStorage.getItem("user"));

        // localStorage.setItem("hii", "hii team")
        // console.log(localStorage.getItem("hii"))
        // localStorage.removeItem("hii");
        // localStorage.clear()


        setTimeout(() => {
            fetch("https://jsonplaceholder.typicode.com/posts")
                .then((res) => res.json())
                .then((data) => {
                    console.log(data)
                    let usersPost = '';
                    data.forEach((datas) => {
                        // usersPost += ` <p ${datas.userId}>
${datas.title} </p> `
                        // usersPost += ` <li ${datas.userId}>
${datas.title} </li> `
                        usersPost += `
                                        <tr ${datas.id}>
                                            <td> ${datas.id} </td>
                                            <td > ${datas.title} </td>
                                        </tr>
                                    `
                        document.getElementById("datas").innerHTML =
usersPost;
                    })
                })
```

```
                    .catch((e) => { console.error(e) })
            }, 5000)

        </script>
    </section>

    <footer>
        <div class="footerbox">
            <img src="assets/logo.png" alt="">
            <div class="links">
                <ul>
                    <li><a href="#">Home</a></li>
                    <li><a href="#">About</a></li>
                    <li><a href="#">Service</a></li>
                    <li><a href="#">Contact</a></li>
                </ul>
            </div>
            <div class="linkicons">
                <ul>
                    <li><a href="#"><ion-icon
name="logo-facebook"></ion-icon></a></li>
                    <li><a href="#"><ion-icon
name="logo-whatsapp"></ion-icon></a></li>
                    <li><a href="#"><ion-icon
name="mail-open-outline"></ion-icon></a></li>
                    <li><a href="#"><ion-icon
name="logo-instagram"></ion-icon></a></li>
                </ul>
            </div>
            <div>
                <p>All reserved by coworkers.in.org</p>
            </div>
        </div>




    </footer>
```

```
    <script type="module"
src="https://unpkg.com/ionicons@8.0.13/dist/ionicons/ionicons.esm.js"></sc
ript>
    <script nomodule
src="https://unpkg.com/ionicons@8.0.13/dist/ionicons/ionicons.js"></script
>
</body>

</html>
```

Mysql

Source:

https://www.geeksforgeeks.org/sql/mysql-interview-questions/

https://www.simplilearn.com/mysql-interview-questions-article

https://netmaxtech.com/mysql-interview-questions-answers/

https://www.w3schools.com/MySQL/mysql_quiz.asp

1. **What is the difference between MySQL and SQL?**
2. **What are the different data types available in MySQL?**
3. **What is a primary key? Can a table have multiple primary keys?**
4. **What is the difference between PRIMARY KEY and UNIQUE constraint?**
5. **Explain the difference kind of constrained in mysql**
6. **What is a foreign key in MySQL?**

7. **How do you retrieve the current date and time in MySQL?**

8. **What is the default port number for MySQL?**

9. **What are aggregate functions in MySQL? Can you name a few?**

10. **What is the difference between WHERE and HAVING clauses?**

1. **What is the difference between INNER JOIN and LEFT JOIN?**

2. **How can you eliminate duplicate rows in a query result?**

3. **What is the use of the GROUP BY clause?**

4. **What is the LIMIT clause used for?**

1. **How do you create a table in MySQL?**

2. **What is the difference between TRUNCATE, DELETE, and DROP?**

Basic Level

1. **What is MySQL?**

2. **What are the data types in MySQL?**

3. **Difference between CHAR and VARCHAR?**

4. **What is the primary key and foreign key?**

5. **How do you create a table in MySQL?**

6. **How to insert multiple rows into a table?**

7. **What is the difference between WHERE and HAVING?**

8. **What is the use of LIMIT clause in MySQL?**

9. **What is normalization? What are the types?**

10. **What is a unique key? Can a table have multiple unique keys?**

🔹 Intermediate Level

1. **What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?**
2. **What is a subquery? Give an example.**
3. **How does GROUP BY work? How does it differ from ORDER BY?**
4. **How to find the second highest salary from a table?**
5. **What are indexes? How do they improve performance?**
6. **What is the default storage engine in MySQL?**
7. **What is the difference between NOW() and CURRENT_TIMESTAMP?**
8. **What are views? How do you create one?**
9. **What is the difference between DELETE, TRUNCATE, and DROP?**
10. **How do you prevent SQL injection in MySQL queries?**

🔹 Advanced Level

1. **What is the ACID property in MySQL? Explain each component.**
2. **How do transactions work in MySQL?**
3. **What are stored procedures? How do you use them?**
4. **Explain triggers in MySQL with an example.**
5. **What are the differences between InnoDB and MyISAM?**
6. **What are temporary tables and when to use them?**
7. **How can you optimize a slow-running query?**
8. **Explain the use of EXPLAIN or EXPLAIN PLAN in MySQL.**
9. **What is replication in MySQL? Types of replication?**

10. **How do you handle database locking and concurrency in MySQL?**

🔹 Practical & Real-time Scenario-Based Questions

1. **How would you design a table for storing employee records?**
2. **Write a query to get department-wise maximum salary.**
3. **How would you retrieve duplicate records from a table?**
4. **How would you design a query to paginate results (10 per page)?**
5. **If a user reports data inconsistency, how do you debug it?**
6. **Write a query to get top 3 highest-paid employees per department.**
7. **How do you implement audit logging using triggers in MySQL?**
8. **How do you backup and restore a MySQL database?**
9. **What's your strategy for designing a scalable database?**
10. **How to write a stored procedure to calculate bonuses for employees?**

Html

https://www.geeksforgeeks.org/html/html-interview-questions/

https://www.simplilearn.com/html-interview-questions-and-answers-article

https://www.interviewbit.com/html-interview-questions/

Css

https://www.geeksforgeeks.org/css/css-interview-questions/
https://www.interviewbit.com/css-interview-questions/
https://www.simplilearn.com/tutorials/css-tutorial/css-interview-questions

Js

https://roadmap.sh/questions/javascript

https://www.geeksforgeeks.org/javascript/javascript-interview-questions/
https://www.interviewbit.com/javascript-interview-questions/

Html,Css,Js coding questions

Html

1. **Create a simple registration form with fields: name, email, password, and a submit button.**
   ➤ Validate that all fields are filled before submission.
   (Use HTML form + `required` attribute)
2. **Create a table with 3 columns — Name, Email, and Phone.**
   ➤ Add a border and alternate row background color.
3. **Add an image that redirects to another website when clicked.**
4. **Create an unordered and ordered list showing your favorite movies.**
5. **Create a navigation bar with links to Home, About, Services, and Contact.**
6. **Design a semantic layout using `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>`.**
7. **Embed a video and audio in a webpage with controls.**
8. **Create a form with `radio` buttons for gender selection and checkboxes for skills.**

CSS

1. **Build a login form using HTML5 validation (`pattern`, `email`, `minlength`).**
2. **Create a responsive image gallery using `<figure>` and `<figcaption>`.**
3. **Center a `div` both horizontally and vertically using Flexbox.**
4. **Change background color of a button when hovered.**
5. **Create a box with rounded corners, shadow, and gradient background.**

6. **Write CSS to make text uppercase, bold, and underlined.**
7. **Create a responsive 3-column layout that becomes 1 column on small screens.**
8. **Create a sticky navigation bar that stays on top when scrolling.**
9. **Build a CSS-only dropdown menu.**
10. **Create a loading spinner animation using keyframes.**
11. **Create a tooltip that appears when hovering over a button.**
12. **Build a responsive card design with hover effects.**

js

1. **Swap two variables without using a third variable.**
2. **Check if a number is even or odd.**
3. **Reverse a string using JavaScript.**
4. **Count vowels in a string.**
5. **Find maximum number in an array.**
6. **Print a 2D matrix row and column wise.**
7. **Write a function to check if a string is a palindrome.**
8. **Create a simple digital clock using `setInterval()`.**
9. **Fetch user data from `https://jsonplaceholder.typicode.com/users` and display names in console.**
10. **Create a counter app with "+" and "–" buttons that update count dynamically on screen.**
11. **Build a theme toggler (Light/Dark mode) using JavaScript.**
12. **Create a form validation system that checks:**
13. **Empty fields**
14. **Valid email format**
15. **Password length ≥ 6**
16. **Build a simple To-Do app using HTML, CSS, JS (add & delete tasks).**

17. **Implement image slider using `setInterval()` or `setTimeout()`.**
18. **Store and retrieve data from `localStorage`.**
19. **Combined (HTML + CSS + JS) Mini Projects**
20. **Registration Form with JS Validation**
21. **Login Page with Show/Hide Password**
22. **Theme Toggle Button (Dark/Light)**
23. **To-Do List App**
24. **Simple Calculator**
25. **Popup Modal (Open/Close)**
26. **Image Carousel / Slider**
27. **Form Data Save in LocalStorage**
28. **Responsive Navbar Toggle**
29. **Stopwatch / Timer**

Reactjs

https://www.interviewbit.com/react-interview-questions/

https://www.geeksforgeeks.org/reactjs/react-interview-questions/

https://www.simplilearn.com/tutorials/reactjs-tutorial/reactjs-interview-questions

React Js Test Questions

1. What is React? How is it different from traditional JavaScript frameworks?
2. What is the virtual DOM and how does React use it?
3. What are the main features of React?
4. What is JSX? Can a browser read JSX directly?
5. What are components in React? Write basics syntax
6. What is the purpose of `key` in lists?
7. How do you pass data from parent to child in React?

8. What is the difference between **state** and **props**?
9. What is a Hook in React?
10. What are the main lifecycle methods in a class component?

11.List out the Major and main concepts in Reactjs(Key Words)

1. Introduction & Fundamentals

1. **What is React.js?**
2. **What are the main features of React?**
3. **What is the difference between a class component and a functional component?**
4. **What is JSX and why is it used?**
5. **What are components in React?**
6. **What is the difference between state and props?**
7. **What is virtual DOM and how does it work?**
8. **What is the purpose of key in React lists?**
9. **Explain how React updates the UI efficiently.**
10. **What are React Fragments and why are they used?**

---

2. React Hooks

1. **What are hooks in React?**
2. **Explain `useState()` with an example.**

3. **Explain `useEffect()` with an example.**
4. **What is the difference between `useEffect()` and lifecycle methods?**
5. **What is `useRef()` used for?**
6. **What is `useMemo()` and when to use it?**
7. **What is `useCallback()` and how is it different from `useMemo()`?**
   ///
8. **How to create a custom hook?**

---

3. Props, State & Lifecycle

1. **How do you pass data from parent to child components?**
2. **How can you pass data from child to parent components?**
3. **What are controlled and uncontrolled components?**
4. **Explain component lifecycle methods in class components (`componentDidMount`, `componentDidUpdate`, etc.).**
5. **What is lifting state up in React?**

---

4. Rendering & Events

1. **What is conditional rendering in React?**
2. **What is list rendering?**
3. **How to handle forms in React?**
4. **How do you handle events in React?**
5. **What are synthetic events in React?**

---

5. Routing

1. **What is React Router?**
2. **How to implement routing in React?**
3. **What is `<Link>` vs `<a>` tag?**
4. **What is a route parameter?**
5. **What is `useNavigate()` in React Router v6?**

---

6. State Management

1. **What is Redux?**
2. **What are the main principles of Redux?**
3. **What are actions, reducers, and store?**
4. **How do you connect Redux with React?**
5. **What is Redux Toolkit and why is it preferred?**
6. **What is the difference between Context API and Redux?**
7. **Create a simple functional component that displays “Hello, React!”**
8. **Create a component with a button that shows an alert when clicked.**
9. **Create a component that displays your name using props.**
10. **Create a counter using useState().**
11. **Create a simple input box that updates and displays text in real-time**
12. **Fetch API data and display it using useEffect().**
13. **Create a theme toggler (dark/light) using state.**
14. **Create a form with name and email, and show submitted data below it**

15. **Create a To-Do List App with add and delete functionality.**
16. **Create a conditional rendering component.**
17. **Use useRef() to focus input on button click.**
18. **Use useContext() to share theme between components.**
19. **Build a custom hook that logs window width (responsive tracking).**
20. **Create a data table with sorting functionality.**
21. **Build a CRUD app (Add, Edit, Delete user) with mock data.**
22. **Login form with validation + show/hide password.**
23. **Dashboard page showing users in a table using `axios`.**
24. **Search filter for a list of names.**
25. **Responsive navigation bar with menu toggle on mobile.**
26. **Image slider using `useState` and `setInterval`.**
27. **Form data stored in LocalStorage.**
28. **React + Node.js integration (Frontend calls backend API).**
29. **Redux counter example using Redux Toolkit.**
30. **React Router setup for navigation between pages.**